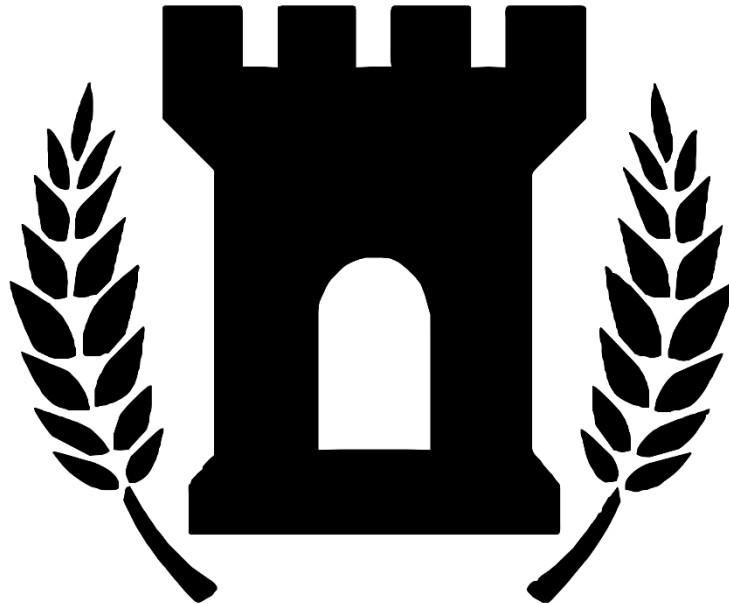




Feldfeste Operation Instruction



Feldfeste Games Studio

FF-OI-11: "Damage system"

Document History

Revision	Content	Editor	Time
01	Initial Document Creation	Arina	23.06.2026



Table of Contents

Overview.....	3
CA Damage Type.....	4
Implemented DamageTypes.....	4
Damage Delivery Type.....	5
Piercing damage.....	6
Damage Dealer Interface.....	7
Status Effects.....	9
When adding a new effect.....	9
Implemented Effects.....	9
Status Effect Target Interface.....	10
Defense component.....	11
Damage calculation.....	12
Stats involved in damage calculation:.....	12
Damage pipeline.....	12
Effect Damage.....	14
Kinetic impact.....	15
Attack speed.....	16
Anim notify.....	17



Overview

This document describes the damage and status effect system architecture. The system is designed to be data-driven and extensible — adding new damage types or effects requires no changes to existing code.

Damage is defined by two independent dimensions: **what kind** (`CA_DamageType`) and **how it is delivered** (`E_CA_DamageDeliveryType`). All damage calculation happens in `HealthComponent` — via `OnTakeDamage` for incoming attacks and `ApplyEffectDamage` for effect tick damage. Delivery type and piercing data are retrieved from the `DamageCauser` via the `Damage Dealer Interface`.

Status effects are self-contained ActorComponents added dynamically to the target. The `Status Effect Component` acts as a manager — it creates, tracks, and removes effects. Each effect handles its own behaviour and lifetime independently.

Defensive stats live in the `Defense Component`, which provides resist and block/evade modifiers as simple lookups. Final damage is calculated from these modifiers with no branching logic.

Actors that can receive status effects implement the `Status Effect Target Interface` to handle visual and gameplay reactions without coupling to specific effect classes.

Key Concepts

- `CA_DamageType` — defines the nature of damage (Physical, Fire)
- `E_CA_DamageDeliveryType` — enum defining how damage is delivered (Melee, SmallProjectile, LargeProjectile)
- `Damage Dealer Interface` — implemented by `DamageCausers` to expose `DeliveryType` and `Piercing`
- `Status Effect Component` — manager component that creates, tracks, and removes active effects on an actor
- `Status Effects` — self-contained ActorComponents defining the behaviour of a specific effect
- `StatusEffectTarget Interface` — implemented by actors to react to effect application and expiration without casts
- `Defense Component` — holds defensive stats; provides resist and block/evade modifiers as Map lookups
- `Damage Calculation` — the pipeline that resolves incoming damage into a final value using attacker stats, defensive modifiers, and damage type.



CA Damage Type

Base class located:

[Content/CA_Code/Blueprints/DamageType/CA_DamageTypeBase.uasset](#)

Defines the nature of damage. Used as a key for resist lookup in DefenseComponent.

Properties

- **StatusEffectClass** — optional. The effect to apply on hit.

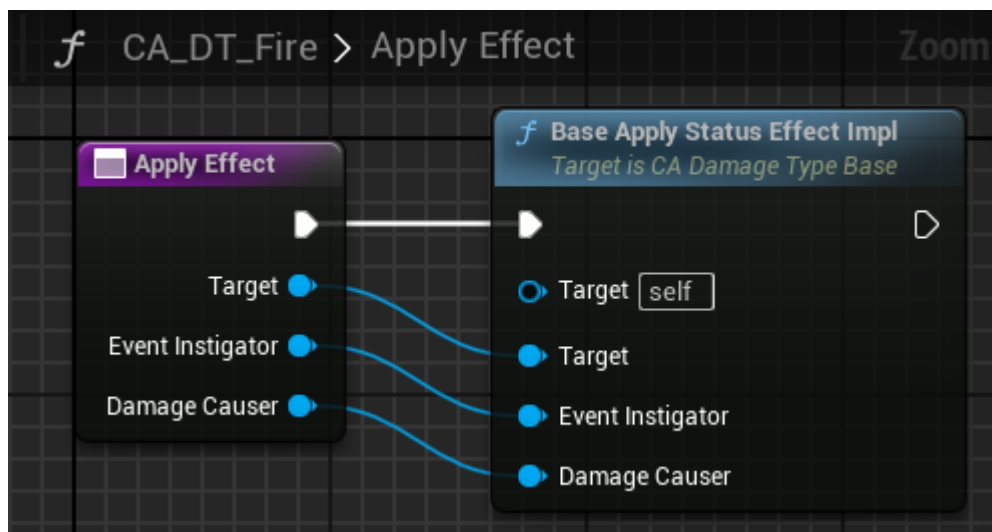
Functions

- **ApplyEffect** (*public*) — called in the damage pipeline. Override this in every subclass to define what happens on hit. Base implementation does nothing.
- **BaseApplyStatusEffectImpl** (*protected*) — helper implementation that passes StatusEffectClass to the target's StatusEffectManagerComponent. Call this from ApplyEffect if the damage type applies a status effect.

Adding a new DamageType

Create a subclass and override ApplyEffect. Without an override no effect will be applied.

Example: CA_DT_Fire overrides ApplyEffect and calls BaseApplyStatusEffectImpl to apply burn.



Implemented DamageTypes

- **CA_DT_Fire** — overrides [ApplyEffect](#), attempts to apply Burn via [BaseApplyStatusEffectImpl](#)
- **CA_DT_Physical** — no additional effect on hit



Damage Delivery Type

Located:

[Content/CA_Code/Blueprints/DamageType/E_CA_DamageDeliveryType](#)

Enum defining how damage is delivered:

- Melee
- RangedSmallProjectile
- RangedLargeProjectile.

Delivery type is intentionally separate from [CA_DamageType](#) — the same damage type can be delivered in different ways (e.g. Fire can be both a melee weapon and an arrow projectile). Merging them would require a separate DamageType class per combination, which does not scale.

Since Unreal's [ApplyDamage](#) does not support custom parameters, delivery type is stored on the [DamageCauser](#) and retrieved via the [Damage Dealer Interface](#) inside the damage pipeline.

Each delivery type is used as a key for block/evade chance lookup inside the [DefenseComponent](#).



Piercing damage

A portion of base damage that **bypasses block entirely**.

Calculated as $\text{BaseDamage} \times \text{PiercingPercent}$, where **PiercingPercent** is a value from 0 to 1.

Retrieved via **BPI_DamageDealer**. For master units, stored in **BattleComponent**.

If block succeeds, only the piercing portion is dealt. The non-piercing remainder is blocked.



Damage Dealer Interface

Located: [Content/CA_Code/Blueprints/Interfaces/BPI_DamageDealer](#)

Implemented by actors that deal damage (projectiles, melee weapons, etc.). Provides the damage pipeline with data that cannot be passed through Unreal's [ApplyDamage](#) directly.

Functions

- [GetDamageDeliveryType](#) (*public*) — returns [E_CA_DamageDeliveryType](#) of the causer
- [GetPiercingPercent](#) (*public*) — returns piercing value; reduces block chance on the target
- [GetBaseDamage](#) (*public*) — returns base damage information

Usage

Called inside [HealthComponent](#) during [OnTakeDamage](#) to retrieve delivery and piercing data from the [DamageCauser](#). If the causer does not implement this interface, default values are used and damage is processed without delivery modifiers.



Status Effect Manager Component

Located:

CA_Code/Blueprints/ActorsComponents/StatusEffectComponent/StatusEffectsManagerComponent

Manager of active status effects on an actor. Responsible for creating, tracking, reapplying, and removing effect instances. Broadcasts events on effect applied, reapplied, and expired.

⚠ Effects must only be removed through this manager. Since *ActorComponents* can only be destroyed by their owning actor, removal is handled via *RemoveEffect* from *BPI_StatusEffectTarget*. Never call *DestroyComponent* on an effect directly.

Public interface — querying active effects (count, classes, data, immunity check) and applying or manually removing effects via *ApplyEffect* and *Manual Removal Request*.

Protected — all internal management logic: creating instances, handling removal, reapplication, and destruction.

Instance Events — internal handlers for effect lifecycle (applied, reapplied, expired). Bound automatically when a new effect is created

Variables

- *ImmuneToAll* (bool) — if true, the actor is immune to all status effects. *ApplyEffect* will do nothing regardless of effect type.
- *ResistedEffectTypes* (Set) — a set of effect classes the actor is immune to.

Immunity is resolved automatically inside *ApplyEffect*. No external checks are needed before calling it.



Status Effects

CA_StatusEffectBase

Located:

CA_Code/Blueprints/ActorsComponents/StatusEffectComponent/StatusEffects/CA_StatusEffectBase

Base class for all status effects. Effects are ActorComponents added dynamically to the target by `StatusEffectsManagerComponent`. An effect can have a limited lifetime or run indefinitely. Damage is optional but supported at the base level.

Public variables

- `LifetimeEffect` (*public, Float*) — duration of the effect in seconds
- `Infinite` (*public, Bool*) — if true, the effect never expires on its own
- `DamageType` (*public*) — damage type applied by this effect

CA_StatusEffect_PeriodicBase

Located:

CA_Code/Blueprints/ActorsComponents/StatusEffectComponent/StatusEffects/CA_StatusEffect_PeriodicBase

Extends `CA_StatusEffectBase` for effects that periodically affect the target.

Public variables

- `InitStartDelay` (*public, Float*) — delay before the first tick fires
- `TimerTickInterval` (*public, Float*) — interval between ticks in seconds
- `LoopingTickTimer` (*public, Bool*) — whether the tick repeats or fires once

When adding a new effect

Override `ApplyOnOwner`, `Reapply`, and `Expire` from the base class. Always call the parent implementation unless you explicitly need to replace the base behaviour — the base functions handle timer management and event broadcasting. If you skip the parent call, make sure to fire the relevant events manually and review the base logic carefully before doing so.

Implemented Effects

- `CA_BurnEffect` — infinite periodic effect that deals fire damage to the owner at each tick. Must be manually removed by extinguishing the actor via `StatusEffectsManagerComponent`.



Status Effect Target Interface

BPI_StatusEffectTarget

Located: [CA_Code/Blueprints/Interfaces/BPI_StatusEffectTarget](#)

Implemented by actors that can receive status effects.

At first glance redundant given [StatusEffectsManagerComponent](#), but serves two distinct purposes:

Effect removal — ActorComponents can only be destroyed by their owning actor. [RemoveEffect](#) is the only way to correctly handle destruction without casts or direct [DestroyComponent](#) calls from outside.

Actor reactions — functions like [OnStartBurn](#) / [OnEndBurn](#) allow each actor to define its own visual and audio response to effects without the effect class knowing anything about the actor. This avoids casts and if/else chains — the effect simply calls the interface, each actor implements its own reaction independently.

These reaction functions are primarily for visuals and sound, but can also drive behaviour changes on the actor. If an actor has no reaction to a specific effect — simply leave the interface functions unimplemented. No stub or empty implementation required.

This interface will grow with every new effect added — each new effect type requires a corresponding pair of functions ([OnStart](#) / [OnEnd](#)). This is an accepted **trade-off for readability and decoupling**. When adding a new effect, remember to extend this interface accordingly.



Defense component

Located: `/CA_Code/Blueprints/ActorsComponents/DefenseComponent.uasset`

Holds all defensive stats for the unit. Provides block, evade, and damage resistance values used in the damage calculation pipeline.

All parameters are clamped between 0 and 1.

Variables

Base values are set per unit and exposed for configuration. Additional values are applied at runtime via buffs and debuffs and are not directly editable.

- `BaseDamageBlockRate` (*public, read only*) — `Map<DeliveryType, float>`
- `BaseDamageEvadeRate` (*public, read only*) — `Map<DeliveryType, float>`
- `BaseDamageResistances` (*public, read only*) — `Map<DamageType, float>`
- `AdditionalBlockRate` (*private*) — runtime modifier, applied on top of base
- `AdditionalEvadeRate` (*private*) — runtime modifier, applied on top of base
- `AdditionalDamageResistance` (*private*) — runtime modifier, applied on top of base

Modifiers — public functions to apply and remove block, evade, and damage modifiers at runtime (buffs/debuffs). `RemoveAllModifiers` clears everything at once.

Getters — public functions to retrieve final computed rates. Always use getters in the damage pipeline — they account for both base and additional values.



Damage calculation

Stats involved in damage calculation:

Attacker stats — located in `BattleComponent`:

- **BaseDamage** — unit's damage
- **Accuracy** — affects hit chance
- **DamageType** — nature of the attack
- **DamageDeliveryType** — how the damage is delivered
- **PiercingPercent** — portion of damage that bypasses block

Attack speed, attack range and other stats exist on the unit but do not participate directly in damage calculation.

Target stats — located in `DefenseComponent`:

- **BlockRate** — block coefficient per delivery type.
- **EvadeRate** — evade coefficient per delivery type.
- **DamageResistance** — damage reduction per damage type.

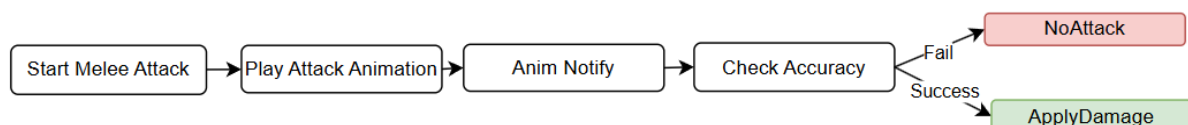
All three defensive stats are composed of a base value plus an additional modifier from buffs or debuffs.

Damage pipeline

Attack side

Attacks are executed via **Behavior Tree** tasks located at `CA_Code/Blueprints/AI/Tasks/Battle/Attacks/`. `BTT_BaseAttack` is the base class — current implementations are melee and ranged.

- **Melee** — applies damage directly. Attack lands only if **Accuracy** check passes.
Base flow:

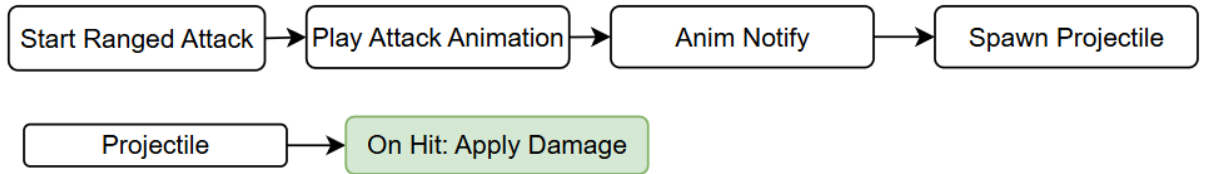


- **Ranged** — spawns a projectile. Accuracy does not apply — the hit is determined by whether the projectile reaches the target. On hit, the projectile applies damage.

Base flow:



FF-OI-11: "Damage system" Rev(01)



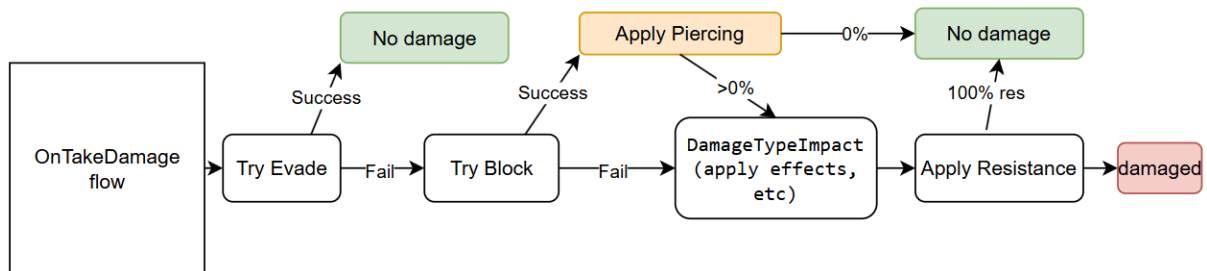
The attacker is only responsible for initiating the attack and providing its stats.

Defense side

Evade, block, and final damage calculation happen on the defending actor inside **HealthComponent**.

Damage is applied via Unreal's **ApplyDamage**, executed on the server. **HealthComponent** subscribes to this and handles all calculation and event broadcasting.

Defense flow:



For full details see **HealthComponent**

***PiercingPercent** and **DeliveryType** are not passed as parameters into **ApplyDamage** — they are retrieved from the **DamageCauser** via **BPI_DamageDealer** inside **OnTakeDamage**.*

*This is a known limitation of using Unreal's **ApplyDamage** directly — it does not support custom parameters beyond **DamageTypeClass**. The decision was made to keep the standard pipeline intact rather than introduce a custom damage function for primary damage.*

***DamageCauser** is expected to be valid at the moment of the call. Since **ApplyDamage** is synchronous, stats are retrieved in the same frame the attack lands — before the causer could become invalid. All causer references are validity-checked before use. In practice this holds, but there is a theoretical edge case where the causer is destroyed before the call completes. This is considered an acceptable risk given the synchronous nature of the pipeline.*



FF-OI-11: “Damage system” Rev(01)

Effect Damage

Status effects deal damage through a separate server event `ApplyEffectDamage` in `HealthComponent`, bypassing the main `OnTakeDamage` pipeline. This avoids an effect reapplying itself on each damage tick.

Effect damage cannot be evaded or blocked — only `DamageResistance` is applied before dealing the final damage.



Kinetic impact

This section is under development.



Attack speed

This section is under development.



FF-OI-11: “Damage system” Rev(01)

Anim notify

This section is under development.